Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №3

По дисциплине «Методы обработки изображений»

Выполнил:

Студент группы P3306,

Михайлов Дмитрий Андреевич

Преподаватель:

Потемин Игорь Станиславович

Санкт-Петербург

2026 год

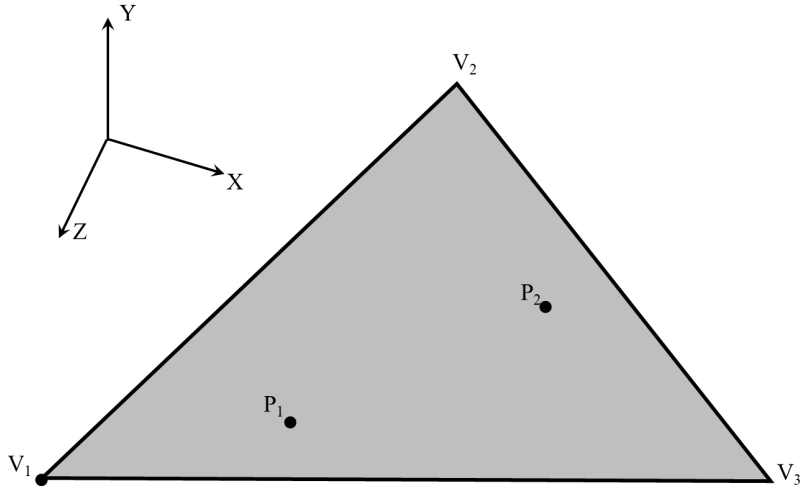
Содержание

Задание 3
Листинг программы 5
Фрагмент 1. Структура трехмерного вектора Vec3 5
Фрагмент 2. Вспомогательные функции общего назначения 7
Фрагмент 3. Построение локального ортонормированного базиса 8
Фрагмент 4. Вывод статистики по областям 9
Фрагмент 5. Генерация случайной точки внутри треугольника 10
Фрагмент 6. Проведение эксперимента для треугольника 11
Фрагмент 7. Генерация случайной точки внутри круга 14
Фрагмент 8. Проведение эксперимента для круга 15
Фрагмент 9. Генерация равномерного направления на единичной сфере 17
Фрагмент 10. Проведение эксперимента для единичной сферы 18
Фрагмент 11. Генерация направления с косинусным распределением относительно нормали 20
Фрагмент 12. Проведение эксперимента для косинусного распределения 21
Фрагмент 13. Главная функция программы 24
Параметры вычислительного эксперимента 25
Результаты вычислений 25
Анализ результатов вычислений 28
Вывод 29

Задание

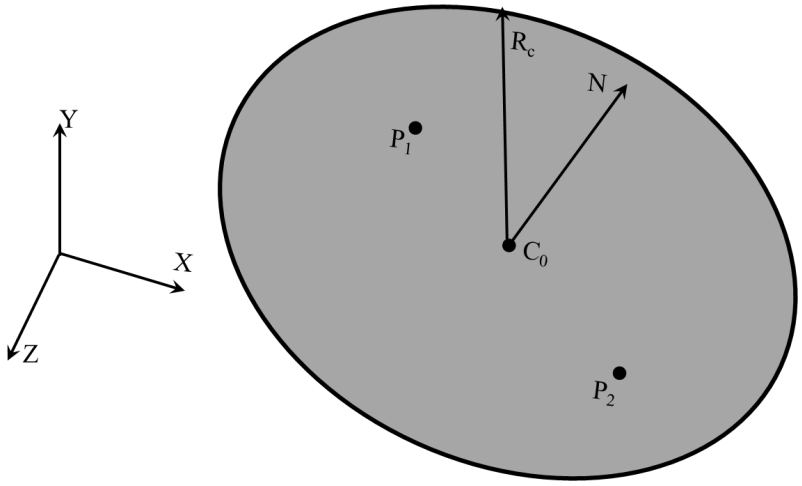
Изучить методы формирования заданных распределений случайных координат и направлений лучей в пространстве. Доказать корректность полученных распределений случайных величин.

1. Сформировать равномерное распределение случайных точек (P_1, P_2, …) внутри треугольника. Треугольник задан тремя координатами в пространстве (V_1, V_2, V_3). Число точек используемых в выборке 100000, число точек, сформированных внутри треугольника также 100000.



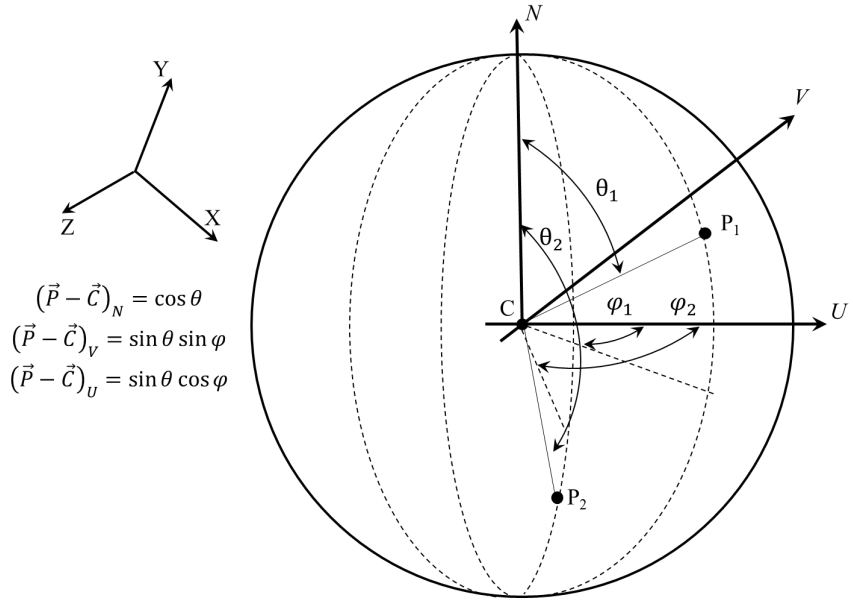
Доказать, что полученное распределение равномерное и все точки лежат внутри треугольника.

2. Сформировать равномерное распределение случайных точек (P_1, P_2, …) внутри круга. Круг задан ориентацией нормали N, радиусом круга R_c, и его центром C. Число точек используемых в выборке 100000, число точек, сформированных внутри круга также 100000.



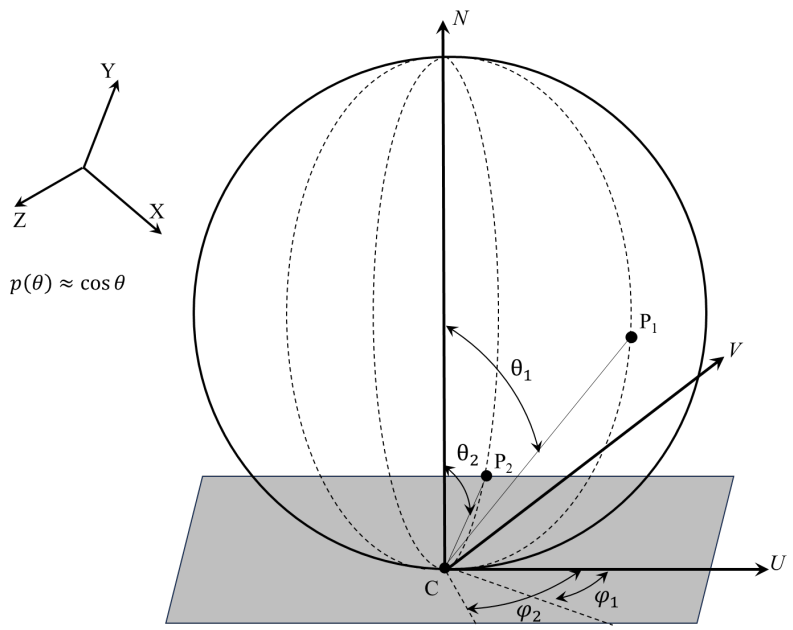
Доказать, что полученное распределение равномерное и все точки лежат внутри круга.

3. Сформировать равномерное распределение случайных направлений в пространстве (точек на единичной сфере P_1, P_2, …). Число точек используемых в выборке 100000, число полученных направлений также 100000.



Доказать, что полученное распределение равномерное и для всех выборок были сформированы направления.

4. Сформировать распределение случайных направлений с косинусным распределением плотности вероятности относительно вектора N (единичных векторов, ориентированных по направлению P_1 – C, P_2 – C, …, плотность вероятности пропорциональна длине вектора P – C). Распределение симметрично относительно вектора N. Число точек используемых в выборке 100000, число полученных направлений также 100000.



Доказать, что полученное распределение косинусное и для всех выборок были сформированы направления.

Листинг программы

Фрагмент 1. Структура трехмерного вектора Vec3

```
import math
import random
from dataclasses import dataclass
from typing import List, Tuple



SAMPLES = 100000
SEED = 42



@dataclass
class Vec3:
    x: float
    y: float
    z: float

    def __add__(self, other: "Vec3") -> "Vec3":
        return Vec3(self.x + other.x, self.y + other.y, self.z + other.z)
```

```
    def __sub__(self, other: "Vec3") -> "Vec3":
        return Vec3(self.x - other.x, self.y - other.y, self.z - other.z)

    def __mul__(self, k: float) -> "Vec3":
        return Vec3(self.x * k, self.y * k, self.z * k)

    def __rmul__(self, k: float) -> "Vec3":
        return self.__mul__(k)

    def __truediv__(self, k: float) -> "Vec3":
        return Vec3(self.x / k, self.y / k, self.z / k)

    def dot(self, other: "Vec3") -> float:
        return self.x * other.x + self.y * other.y + self.z * other.z

    def cross(self, other: "Vec3") -> "Vec3":
        return Vec3(
            self.y * other.z - self.z * other.y,
            self.z * other.x - self.x * other.z,
            self.x * other.y - self.y * other.x
        )

    def norm(self) -> float:
        return math.sqrt(self.dot(self))

    def normalized(self) -> "Vec3":
        n = self.norm()
        if n == 0:
            raise ValueError("Нельзя нормализовать нулевой вектор")
        return self / n

    def __repr__(self) -> str:
        return f"({self.x:.6f}, {self.y:.6f}, {self.z:.6f})"
```

В начале программы подключаются стандартные модули math и random, а также объявляются две константы: SAMPLES = 100000 и SEED = 42. Первая

задает число выборок в каждом вычислительном эксперименте, что соответствует условию задания, а вторая фиксирует начальное состояние генератора случайных чисел, благодаря чему результаты программы остаются воспроизводимыми при повторных запусках.

Далее определяется структура Vec3, представляющая трехмерный вектор с координатами x, y, z. Она используется во всей программе для хранения координат точек, направлений и нормалей. Применение отдельного класса для векторов делает код более наглядным и избавляет от необходимости вручную работать с тройками чисел в каждой формуле.

Методы __add__, __sub__, __mul__, __rmul__ и __truediv__ реализуют основные арифметические операции над векторами: сложение, вычитание, умножение на число и деление на число. Эти операции необходимы, например, при вычислении точек внутри треугольника по барицентрической формуле, при построении точек круга через базисные векторы и при формировании направлений на сфере и полусфере.

Метод dot вычисляет скалярное произведение двух векторов. Оно используется при нахождении длины вектора, а также при проецировании точки или направления на оси локального базиса. Метод cross вычисляет векторное произведение и применяется для построения ортонормированного базиса в плоскости, перпендикулярной заданной нормали. Это особенно важно во втором и четвертом пунктах задания, где нужно работать с кругом и полусферой, ориентированными относительно произвольного вектора N.

Метод norm возвращает длину вектора, а метод normalized строит единичный вектор того же направления. Нормировка требуется в тех местах программы, где необходимо получить единичную нормаль или единичное направление. Дополнительная проверка на нулевую длину защищает программу от некорректной нормировки нулевого вектора. Метод __repr__ задает удобный формат вывода вектора на экран, благодаря чему результаты вычислений отображаются компактно и читаемо.

Фрагмент 2. Вспомогательные функции общего назначения

```
def print_header(title: str) -> None:
    print("\n" + "=" * 100)
    print(title)
    print("=" * 100)



def print_subheader(title: str) -> None:
    print("\n" + "-" * 100)
```

```
    print(title)
    print("-" * 100)



def mean(values: List[float]) -> float:
    return sum(values) / len(values)
```

Функции print_header и print_subheader используются для структурированного вывода результатов программы. Они не участвуют непосредственно в формировании случайных величин, однако делают результаты вычислительного эксперимента более удобными для анализа, поскольку разделяют вывод на крупные блоки и подпункты. Это полезно при проверке каждого из четырех распределений по отдельности.

Функция mean вычисляет среднее арифметическое элементов списка. Она применяется при анализе полученных выборок: например, для проверки средних значений барицентрических координат, средней величины $(r/R)^2$, средней длины вектора и среднего значения zили $\mu = \cos\ \theta$. Сравнение эмпирических средних с теоретически ожидаемыми значениями позволяет подтвердить корректность сформированных распределений.

Фрагмент 3. Построение локального ортонормированного базиса

```
def build_orthonormal_basis(n: Vec3) -> Tuple[Vec3, Vec3, Vec3]:
    n = n.normalized()

    if abs(n.x) < 0.9:
        helper = Vec3(1.0, 0.0, 0.0)
    else:
        helper = Vec3(0.0, 1.0, 0.0)

    u = n.cross(helper).normalized()
    v = n.cross(u).normalized()
    return u, v, n
```

Функция build_orthonormal_basis строит ортонормированный базис (u, v, n)по заданному вектору нормали n. Сначала нормаль приводится к единичной длине. После этого выбирается вспомогательный вектор helper, который не должен быть почти параллелен нормали. Для этого используется простая проверка компоненты n.x: если она не слишком велика, берется ось

Ox, иначе — ось Oy. Такой выбор позволяет избежать вырожденной ситуации, когда векторное произведение становится слишком малым.

Затем первый базисный вектор u вычисляется как нормированное векторное произведение n × helper. Второй базисный вектор v получается как нормированное произведение n × u. В результате формируется правый ортонормированный базис, где u и v лежат в плоскости, перпендикулярной n. Эта функция играет ключевую роль при генерации точек внутри круга и направлений с косинусным распределением, потому что позволяет перейти от локальных координат к глобальным координатам пространства.

Фрагмент 4. Вывод статистики по областям

```
   def print_counts(title: str, counts: List[int]) -> None:
       total = sum(counts)
       expected = total / len(counts)
       print_subheader(title)
       print(f"{'Область':>10} | {'Число':>10} | {'Доля':>12} | {'Откл. от
ожидаемого':>22}")
       print("-" * 65)
       for i, c in enumerate(counts, start=1):
           frac = c / total
           dev = c - expected
           print(f"{i:>10} | {c:>10} | {frac:>12.6f} | {dev:>22.2f}")
       print(f"\nОжидаемое число в каждой области: {expected:.2f}")
```

Функция print_counts предназначена для вывода статистики попадания случайных точек или направлений в несколько областей одинаковой площади или одинаковой вероятности. Сначала вычисляется общее число наблюдений total, затем — ожидаемое число элементов в каждой области expected, которое равно общему числу наблюдений, деленному на количество областей. После этого для каждой области выводятся абсолютное число попаданий, доля от общего количества и отклонение от ожидаемого значения.

Эта функция используется как инструмент численной проверки равномерности или правильности распределения. Если распределение сформировано корректно, то при достаточно большом числе выборок числа попаданий в равновеликие или равновероятные области должны быть примерно одинаковыми. Поэтому print_counts применяется во всех экспериментах: для разбиения треугольника на 4 части, круга на 8 частей, сферы на 8 частей и косинусного распределения на 8 равновероятных областей.

Фрагмент 5. Генерация случайной точки внутри треугольника

```
   def sample_point_in_triangle(v1: Vec3, v2: Vec3, v3: Vec3, rng:
random.Random) -> Tuple[Vec3, Tuple[float, float, float]]:

       r1 = rng.random()

       r2 = rng.random()


       s = math.sqrt(r1)

       l1 = 1.0 - s

       l2 = s * (1.0 - r2)

       l3 = s * r2


       p = v1 * l1 + v2 * l2 + v3 * l3

       return p, (l1, l2, l3)
```

Функция sample_point_in_triangle формирует одну случайную точку внутри треугольника, заданного вершинами v1, v2 и v3. Для этого сначала генерируются две независимые случайные величины r1 и r2, равномерно распределенные на отрезке [0, 1]. Далее выполняется преобразование s = sqrt(r1), после чего вычисляются барицентрические координаты l1, l2 и l3. Сумма этих коэффициентов всегда равна единице, а сами коэффициенты неотрицательны, поэтому построенная точка гарантированно лежит внутри треугольника.

Точка p находится как линейная комбинация вершин треугольника с весами l1, l2 и l3. Использование квадратного корня в формуле является принципиально важным: без него распределение точек было бы неравномерным. Именно такое преобразование обеспечивает равномерное распределение по площади треугольника. Функция возвращает не только точку p, но и найденные барицентрические координаты, так как они далее используются для проверки принадлежности точки треугольнику и для анализа равномерности распределения.

Фрагмент 6. Проведение эксперимента для треугольника

```
def triangle_experiment(rng: random.Random, samples: int) -> None:
    print_header("1. РАВНОМЕРНОЕ РАСПРЕДЕЛЕНИЕ ТОЧЕК ВНУТРИ ТРЕУГОЛЬНИКА")

    v1 = Vec3(0.0, 0.0, 0.0)
    v2 = Vec3(4.0, 1.0, 0.0)
    v3 = Vec3(1.0, 5.0, 2.0)

    print(f"V1 = {v1}")
    print(f"V2 = {v2}")
    print(f"V3 = {v3}")
    print(f"Число выборок = {samples}")

    centroid_theoretical = (v1 + v2 + v3) / 3.0

    sum_x = sum_y = sum_z = 0.0
    inside_count = 0

    bary1 = []
    bary2 = []
    bary3 = []

    counts = [0, 0, 0, 0]

    for _ in range(samples):
        p, (l1, l2, l3) = sample_point_in_triangle(v1, v2, v3, rng)

        sum_x += p.x
        sum_y += p.y
        sum_z += p.z

        bary1.append(l1)
        bary2.append(l2)
        bary3.append(l3)
```

```
            if l1 >= -1e-12 and l2 >= -1e-12 and l3 >= -1e-12 and abs((l1 + l2
+ l3) - 1.0) < 1e-9:
                inside_count += 1

            if l1 >= 0.5:
                counts[0] += 1
            elif l2 >= 0.5:
                counts[1] += 1
            elif l3 >= 0.5:
                counts[2] += 1
            else:
                counts[3] += 1

        centroid_empirical = Vec3(sum_x / samples, sum_y / samples, sum_z /
samples)

        print_subheader("Проверка принадлежности")
        print(f"Точек внутри треугольника: {inside_count} из {samples}")

        print_subheader("Проверка равномерности")
        print("Теоретически для равномерного распределения:")
        print("- центр масс выборки должен быть близок к центроиду
треугольника")
        print("- средние барицентрические координаты должны быть близки к
1/3")
        print("- в 4 равновеликие части должно попадать примерно по 25%
точек")

        print(f"\nТеоретический центроид: {centroid_theoretical}")
        print(f"Эмпирический центроид : {centroid_empirical}")

        print(f"\nСреднее lambda1 = {mean(bary1):.6f} (ожидается ~ 0.333333)")
        print(f"Среднее lambda2 = {mean(bary2):.6f} (ожидается ~ 0.333333)")
        print(f"Среднее lambda3 = {mean(bary3):.6f} (ожидается ~ 0.333333)")

        print_counts("Попадания в 4 равновеликие части треугольника", counts)
```

Функция triangle_experiment проводит полный вычислительный эксперимент для первого пункта задания. В начале задаются координаты

вершин треугольника v1, v2, v3, после чего вычисляется теоретический центроид треугольника как среднее арифметическое его вершин. Этот центроид используется позже как один из критериев проверки равномерности распределения: при равномерном заполнении треугольника центр масс выборки должен быть близок к теоретическому центроиду.

Далее в цикле длиной samples многократно вызывается функция sample_point_in_triangle, которая генерирует очередную случайную точку и ее барицентрические координаты. Во время обхода выборки накапливаются суммы координат точек, чтобы затем вычислить эмпирический центр масс, а также сохраняются значения l1, l2, l3 для анализа их средних значений. Поскольку для равномерного распределения внутри треугольника математическое ожидание каждой барицентрической координаты равно 1/3, сравнение эмпирических средних с этим значением служит дополнительной проверкой корректности метода.

Отдельно выполняется проверка принадлежности точки треугольнику. Для этого контролируется, что все барицентрические координаты неотрицательны и что их сумма равна единице с учетом малой численной погрешности. Если эти условия выполнены, счетчик inside_count увеличивается. Такая проверка подтверждает, что все сформированные точки действительно находятся внутри заданного треугольника.

Для проверки равномерности треугольник условно разбивается на 4 равновеликие части. Это делается через анализ барицентрических координат: если одна из координат больше либо равна 0.5, точка относится к одной из трех угловых областей; в противном случае она попадает в центральную область. Такое разбиение удобно тем, что все 4 части имеют одинаковую площадь, поэтому при равномерном распределении в них должно попадать примерно одинаковое число точек. Полученные значения сохраняются в массиве counts.

После завершения цикла вычисляется эмпирический центроид выборки и выводятся результаты проверки. В частности, программа показывает, сколько точек оказалось внутри треугольника, сравнивает теоретический и эмпирический центроиды, выводит средние значения барицентрических координат и статистику попадания в 4 равновеликие части. Совокупность этих проверок позволяет сделать вывод о корректности построенного равномерного распределения.

Фрагмент 7. Генерация случайной точки внутри круга

```
def sample_point_in_circle(center: Vec3, normal: Vec3, radius: float, rng:
random.Random) -> Tuple[Vec3, float, float]:

    u1 = rng.random()

    u2 = rng.random()


    r = radius * math.sqrt(u1)

    phi = 2.0 * math.pi * u2


    u, v, n = build_orthonormal_basis(normal)


    p = center + u * (r * math.cos(phi)) + v * (r * math.sin(phi))

    return p, r, phi
```

Функция sample_point_in_circle формирует одну случайную точку внутри круга, заданного центром center, нормалью normal и радиусом radius. Сначала генерируются две независимые случайные величины u1 и u2, равномерно распределенные на отрезке $[0, 1]$. На их основе вычисляются полярные координаты точки: радиус r и угол phi. Радиус берется по формуле r = radius * sqrt(u1), а угол — по формуле phi = 2πu2. Использование квадратного корня здесь принципиально важно, так как оно обеспечивает равномерность именно по площади круга. Если бы радиус выбирался просто равномерно на отрезке $[0, R]$, точки распределялись бы плотнее около центра.

Так как круг может быть расположен в пространстве произвольным образом, далее строится локальный ортонормированный базис u, v, n, связанный с его плоскостью. Векторы u и v лежат в плоскости круга, а n совпадает с нормалью. После этого точка p вычисляется как смещение от центра круга вдоль векторов u и v на величины r cos(phi) и r sin(phi) соответственно. Таким образом, функция возвращает не только саму точку, но и ее полярные параметры r и phi, которые затем используются для проверки корректности и равномерности распределения.

Фрагмент 8. Проведение эксперимента для круга

```
def circle_experiment(rng: random.Random, samples: int) -> None:
    print_header("2. РАВНОМЕРНОЕ РАСПРЕДЕЛЕНИЕ ТОЧЕК ВНУТРИ КРУГА")

    center = Vec3(1.0, 2.0, -1.0)
    normal = Vec3(1.0, 2.0, 3.0).normalized()
    radius = 3.0

    print(f"C = {center}")
    print(f"N = {normal}")
    print(f"R = {radius}")
    print(f"Число выборок = {samples}")

    u_axis, v_axis, n_axis = build_orthonormal_basis(normal)

    inside_count = 0
    radial_sq_values = []

    counts = [0] * 8

    split_r = radius / math.sqrt(2.0)

    for _ in range(samples):
        p, r, phi = sample_point_in_circle(center, normal, radius, rng)

        d = p - center
        proj_u = d.dot(u_axis)
        proj_v = d.dot(v_axis)
        proj_n = d.dot(n_axis)
        rr = math.sqrt(proj_u * proj_u + proj_v * proj_v)

        if abs(proj_n) < 1e-9 and rr <= radius + 1e-9:
            inside_count += 1

        radial_sq_values.append((rr / radius) ** 2)
```

```
        angle = math.atan2(proj_v, proj_u)
        if angle < 0:
            angle += 2.0 * math.pi
        sector = int(angle / (math.pi / 2.0))
        ring = 0 if rr < split_r else 1
        idx = ring * 4 + sector
        counts[idx] += 1

    print_subheader("Проверка принадлежности")
    print(f"Точек внутри круга: {inside_count} из {samples}")

    print_subheader("Проверка равномерности")
    print("Теоретически для равномерного распределения в круге:")
    print("- величина (r/R)^2 должна быть равномерной на [0,1]")
    print("- среднее значение (r/R)^2 должно быть близко к 0.5")
    print("- в 8 равновеликих областей должно попадать примерно одинаковое
число точек")

    print(f"\nСреднее (r/R)^2 = {mean(radial_sq_values):.6f} (ожидается ~
0.500000)")
    print_counts("Попадания в 8 равновеликих областей круга", counts)
```

Функция circle_experiment реализует вычислительный эксперимент для второго пункта задания — формирования равномерного распределения случайных точек внутри круга. В начале задаются параметры круга: его центр center, нормаль normal и радиус radius. Нормаль сразу нормируется, чтобы все дальнейшие вычисления в локальной системе координат выполнялись корректно. После этого для заданной нормали строится локальный ортонормированный базис u_axis, v_axis, n_axis, связанный с плоскостью круга.

Далее запускается цикл по числу выборок. На каждой итерации генерируется очередная точка p внутри круга. Затем вычисляется вектор d = p - center, то есть смещение точки относительно центра. Этот вектор раскладывается по локальному базису: находятся его проекции на оси u, v и n. Компонента proj_n показывает, насколько далеко точка отклоняется от плоскости круга по нормали, а величина rr задает расстояние от центра до точки в самой плоскости круга.

Проверка принадлежности круга выполняется следующим образом: точка считается корректной, если ее отклонение от плоскости по нормали

мало по модулю, а расстояние rr не превышает радиус круга с учетом допустимой численной погрешности. Если оба условия выполняются, увеличивается счетчик inside_count. Такая проверка подтверждает, что все сгенерированные точки действительно находятся внутри заданного круга и лежат в его плоскости.

Для анализа равномерности в список radial_sq_values сохраняется величина $(rr/radius)^2$. Теоретически при равномерном распределении точек по площади круга эта величина должна быть равномерно распределена на отрезке $[0, 1]$, а ее среднее значение должно быть близко к 0.5. Это один из основных критериев корректности метода.

Кроме того, круг условно разбивается на 8 равновеликих областей: сначала на 4 угловых сектора по 90°, а затем на 2 кольцевые зоны. Граница между внутренним и внешним кольцом выбирается как split_r = radius / sqrt(2). Такое значение обеспечивает равенство площадей внутреннего круга и внешнего кольца, поскольку площадь круга пропорциональна квадрату радиуса. В сочетании с четырьмя одинаковыми угловыми секторами это дает 8 областей одинаковой площади. Индекс области вычисляется по номеру сектора и номеру кольца, после чего увеличивается соответствующий элемент массива counts.

После завершения цикла программа выводит результаты эксперимента: число точек, лежащих внутри круга, среднее значение $(r/R)^2$и статистику попадания точек в 8 равновеликих областей. Если распределение получено корректно, то все точки должны лежать внутри круга, среднее значение $(r/R)^2$должно быть близко к 0.5, а числа попаданий в области — мало отличаться друг от друга. Это и позволяет сделать вывод о равномерности распределения внутри круга.

Фрагмент 9. Генерация равномерного направления на единичной сфере

```
def sample_uniform_sphere_direction(rng: random.Random) -> Vec3:
    z = 2.0 * rng.random() - 1.0
    phi = 2.0 * math.pi * rng.random()
    r_xy = math.sqrt(max(0.0, 1.0 - z * z))
    x = r_xy * math.cos(phi)
    y = r_xy * math.sin(phi)
    return Vec3(x, y, z)
```

Функция sample_uniform_sphere_direction формирует одно случайное направление, равномерно распределенное по поверхности единичной сферы. Для этого сначала выбирается координата z, равномерно распределенная на отрезке от -1 до 1. Затем независимо выбирается азимутальный угол phi, равномерно распределенный на интервале от 0 до 2π. После этого вычисляется радиус проекции на плоскость Oxy по формуле r_xy = sqrt(1 - z^2), и далее находятся координаты x и y. В результате получается точка на единичной сфере, которая одновременно задает случайное направление в пространстве.

Такой способ является корректным именно для равномерного распределения по поверхности сферы. Если бы выбирались равномерно сферические углы, например полярный угол θ и азимут φ, то точки сгущались бы вблизи полюсов. Здесь же равномерный выбор z обеспечивает одинаковую вероятность попадания в любые горизонтальные полосы одинаковой высоты, а равномерный угол phi отвечает за равномерность по окружности. Поэтому полученное распределение действительно является равномерным на сфере.

Фрагмент 10. Проведение эксперимента для единичной сферы

```
def sphere_experiment(rng: random.Random, samples: int) -> None:
    print_header("3. РАВНОМЕРНОЕ РАСПРЕДЕЛЕНИЕ НАПРАВЛЕНИЙ НА ЕДИНИЧНОЙ СФЕРЕ")

    lengths = []
    z_values = []

    counts = [0] * 8
    all_generated = 0

    for _ in range(samples):
        d = sample_uniform_sphere_direction(rng)
        all_generated += 1

        length = d.norm()
        lengths.append(length)
        z_values.append(d.z)

        phi = math.atan2(d.y, d.x)
        if phi < 0:
            phi += 2.0 * math.pi
```

```
        sector = int(phi / (math.pi / 2.0))
        hemi = 0 if d.z < 0 else 1
        idx = hemi * 4 + sector
        counts[idx] += 1

    print_subheader("Проверка формирования направлений")
    print(f"Сформировано направлений: {all_generated} из {samples}")

    print_subheader("Проверка корректности")
    print("Теоретически для равномерного распределения на сфере:")
    print("- длина каждого вектора должна быть 1")
    print("- координата z должна быть равномерной на [-1,1]")
    print("- в 8 равновеликих областей сферы должно попадать примерно
одинаковое число направлений")

    print(f"\nСредняя длина вектора = {mean(lengths):.6f} (ожидается ~
1.000000)")
    print(f"Среднее z            = {mean(z_values):.6f} (ожидается ~
0.000000)")
    print_counts("Попадания в 8 равновеликих областей сферы", counts)
```

Функция sphere_experiment проводит вычислительный эксперимент для третьего пункта задания — формирования равномерного распределения направлений на единичной сфере. В начале создаются списки lengths и z_values, в которые затем сохраняются длины сформированных векторов и их z-координаты. Также создается массив counts для подсчета числа попаданий в 8 равновеликих областей сферы и переменная all_generated, позволяющая проверить, что для каждой выборки действительно было сформировано направление.

В основном цикле на каждой итерации вызывается функция sample_uniform_sphere_direction, которая возвращает очередной единичный вектор d. После этого счетчик all_generated увеличивается, а длина вектора и его координата z сохраняются для последующего анализа. Проверка длины нужна для подтверждения того, что все сформированные объекты действительно являются направлениями на единичной сфере. Анализ z важен потому, что при равномерном распределении по поверхности сферы величина z должна быть равномерно распределена на интервале [-1,1], а ее среднее значение — стремиться к нулю.

Для дополнительной численной проверки равномерности сфера разбивается на 8 равновеликих областей. Сначала вся сфера делится на две полусферы: нижнюю (z < 0) и верхнюю (z >= 0). Затем каждая полусфера делится на 4 угловых сектора по азимутальному углу phi, каждый шириной 90 градусов. Поскольку координата z формируется равномерно, а внутри каждой полусферы азимутальный угол также распределен равномерно, такое разбиение действительно задает 8 областей равной площади. Номер области вычисляется по номеру полусферы и сектора, а затем увеличивается соответствующий элемент массива counts.

После окончания цикла программа выводит число сформированных направлений, среднюю длину вектора, среднее значение z и статистику попаданий в 8 равновеликих областей. Если метод реализован корректно, то число сформированных направлений должно совпадать с числом выборок, средняя длина должна быть близка к единице, среднее значение z — близко к нулю, а числа попаданий в области — быть примерно одинаковыми. На основании этих критериев делается вывод о корректности равномерного распределения на единичной сфере.

Фрагмент 11. Генерация направления с косинусным распределением относительно нормали

```
def sample_cosine_weighted_direction(normal: Vec3, rng: random.Random) ->
Vec3:

    u1 = rng.random()

    u2 = rng.random()


    r = math.sqrt(u1)

    phi = 2.0 * math.pi * u2


    x = r * math.cos(phi)

    y = r * math.sin(phi)

    z = math.sqrt(max(0.0, 1.0 - u1))


    u_axis, v_axis, n_axis = build_orthonormal_basis(normal)

    world = u_axis * x + v_axis * y + n_axis * z

    return world.normalized()
```

Функция sample_cosine_weighted_direction формирует одно случайное направление с косинусным распределением относительно заданной нормали normal. Сначала генерируются две независимые случайные величины u1 и u2, равномерно распределенные на отрезке [0, 1]. На их основе в локальной

системе координат строится точка на единичной полусфере. Для этого вычисляются величины r и phi, задающие положение точки на единичном диске, а затем по ним находятся координаты x, y и z. При этом z рассчитывается как sqrt(1 - u1), благодаря чему выполняется условие принадлежности единичной полусфере.

Такой способ построения не дает равномерное распределение по полусфере, а обеспечивает именно плотность вероятности, пропорциональную косинусу угла между направлением и нормалью. Это означает, что направления, близкие к нормали, возникают чаще, чем направления, близкие к касательной плоскости. Подобное распределение широко используется в задачах моделирования отраженного света и глобальной освещенности.

После вычисления локального направления строится ортонормированный базис, связанный с вектором normal. Затем локальный вектор переводится в мировую систему координат с помощью линейной комбинации базисных векторов u_axis, v_axis и n_axis. В конце выполняется нормировка результата, чтобы гарантировать единичную длину сформированного направления. Таким образом, функция возвращает корректный единичный вектор, принадлежащий полусфере, ориентированной относительно заданной нормали.

Фрагмент 12. Проведение эксперимента для косинусного распределения

```
def cosine_experiment(rng: random.Random, samples: int) -> None:
    print_header("4. КОСИНУСНОЕ РАСПРЕДЕЛЕНИЕ НАПРАВЛЕНИЙ ОТНОСИТЕЛЬНО N")

    normal = Vec3(0.0, 0.0, 1.0).normalized()
    print(f"N = {normal}")
    print(f"Число выборок = {samples}")

    mu_values = []
    lengths = []
    positive_count = 0

    counts = [0] * 8

    u_axis, v_axis, n_axis = build_orthonormal_basis(normal)

    for _ in range(samples):
```

```
        d = sample_cosine_weighted_direction(normal, rng)
        lengths.append(d.norm())

        mu = d.dot(n_axis)
        mu_values.append(mu)

        if mu >= -1e-12:
            positive_count += 1

        x = d.dot(u_axis)
        y = d.dot(v_axis)
        phi = math.atan2(y, x)
        if phi < 0:
            phi += 2.0 * math.pi

        sector = int(phi / (math.pi / 2.0))
        t = mu * mu
        layer = 0 if t < 0.5 else 1
        idx = layer * 4 + sector
        counts[idx] += 1

    print_subheader("Проверка формирования направлений")
    print(f"Сформировано направлений: {samples} из {samples}")
    print(f"Направлений в полусфере N: {positive_count} из {samples}")

    print_subheader("Проверка косинусного распределения")
    print("Теоретически для косинусного распределения:")
    print("- все направления должны лежать в полусфере относительно N")
    print("- длина каждого вектора должна быть 1")
    print("- mu = cos(theta) должно иметь плотность 2*mu на [0,1]")
    print("- значит среднее mu должно быть близко к 2/3")
    print("- величина mu^2 должна быть равномерной на [0,1]")
    print("- в 8 равновероятных областей должно попадать примерно одинаковое
число направлений")

    mu_sq_values = [m * m for m in mu_values]
```

```
    print(f"\nСредняя длина вектора = {mean(lengths):.6f} (ожидается ~
1.000000)")

    print(f"Среднее mu=cos(theta) = {mean(mu_values):.6f} (ожидается ~
0.666667)")

    print(f"Среднее mu^2          = {mean(mu_sq_values):.6f} (ожидается ~
0.500000)")


    print_counts("Попадания в 8 равновероятных областей косинусного
распределения", counts)
```

Функция cosine_experiment проводит вычислительный эксперимент для четвертого пункта задания — формирования направлений с косинусным распределением относительно вектора нормали N. В начале задается нормаль normal, для которой в данной реализации выбран вектор (0, 0, 1). Далее создаются списки mu_values и lengths, в которых накапливаются значения косинусов угла между направлением и нормалью, а также длины сформированных векторов. Дополнительно вводится счетчик positive_count, необходимый для проверки того, что все направления действительно лежат в полусфере, ориентированной относительно N.

Внутри цикла на каждой итерации генерируется очередное направление d с помощью функции sample_cosine_weighted_direction. Затем вычисляется его длина и значение mu, равное скалярному произведению направления на единичную нормаль. Поскольку mu = cos(theta), где theta — угол между направлением и нормалью, эта величина является основным параметром для анализа косинусного распределения. Если направление принадлежит правильной полусфере, то mu должно быть неотрицательным. Именно это условие проверяется с помощью счетчика positive_count.

Для проверки формы распределения используется несколько критериев. Во-первых, при косинусном распределении плотность вероятности величины mu на отрезке $[0, 1]$должна быть пропорциональна 2mu, а значит ее математическое ожидание равно $2/3$. Поэтому программа вычисляет среднее значение mu и сравнивает его с теоретическим значением 0.666667. Во-вторых, величина mu^2 при таком распределении должна быть равномерной на $[0, 1]$, следовательно, ее среднее значение должно быть близко к 0.5. Для этого после завершения цикла строится список mu_sq_values, и для него также вычисляется среднее.

Кроме того, для численной проверки корректности распределения полусфера разбивается на 8 равновероятных областей. Для этого азимутальный угол phi делит полусферу на 4 одинаковых сектора, а величина t = mu^2 делит ее на 2 слоя равной вероятности. Поскольку mu^2 при

косинусном распределении должна быть равномерной на [0, 1], разделение по условию t < 0.5 и t >= 0.5 действительно задает два равновероятных слоя. В сочетании с четырьмя одинаковыми азимутальными секторами это дает 8 областей, в которые должно попадать примерно одинаковое число направлений.

После завершения эксперимента программа выводит количество сформированных направлений, число направлений в полусфере относительно N, среднюю длину вектора, среднее значение mu, среднее значение mu^2 и статистику попаданий в 8 равновероятных областей. Если все показатели близки к теоретическим, можно сделать вывод, что реализованный метод действительно формирует корректное косинусное распределение направлений.

Фрагмент 13. Главная функция программы

```
def main() -> None:
    rng = random.Random(SEED)

    print(f"Число выборок для каждого эксперимента: {SAMPLES}")
    print(f"Seed = {SEED}")

    triangle_experiment(rng, SAMPLES)
    circle_experiment(rng, SAMPLES)
    sphere_experiment(rng, SAMPLES)
    cosine_experiment(rng, SAMPLES)



if __name__ == "__main__":
    main()
```

Функция main является точкой входа в программу. В ней создается объект rng - генератор случайных чисел с фиксированным начальным значением SEED. Это позволяет получать воспроизводимые результаты вычислительного эксперимента при каждом запуске программы. Затем на экран выводятся общие параметры эксперимента: число выборок и значение seed.

После этого последовательно вызываются четыре функции: triangle_experiment, circle_experiment, sphere_experiment и cosine_experiment. Каждая из них отвечает за выполнение одного пункта лабораторного задания. Такое разбиение программы на независимые логические блоки делает код более понятным, удобным для тестирования и удобным для анализа

результатов. В конце конструкции if __name__ == "__main__": обеспечивают запуск функции main только при непосредственном выполнении файла как основной программы.

Параметры вычислительного эксперимента

Во всех экспериментах использовалось одинаковое число выборок — **100000**. Для воспроизводимости результатов генератор случайных чисел инициализировался значением **SEED = 42**.

В первом эксперименте равномерное распределение формировалось внутри треугольника с вершинами
$V_1 = (0,0,0)$, $V_2 = (4,1,0)$, $V_3 = (1,5,2)$.

Во втором эксперименте равномерное распределение строилось внутри круга с центром
$C = (1,2,-1)$, нормалью $N = (1,2,3)$и радиусом $R = 3$. Перед вычислениями нормаль приводилась к единичной длине.

В третьем эксперименте формировались **100000** случайных направлений, равномерно распределенных на единичной сфере.

В четвертом эксперименте формировались **100000** направлений с косинусным распределением относительно нормали
$N = (0,0,1)$.

Результаты вычислений

В результате выполнения программы были получены следующие численные результаты. Во всех четырех экспериментах число сформированных точек или направлений составило **100000**, что соответствует условию задания. Программа завершила работу корректно, без ошибок.

1. Равномерное распределение точек внутри треугольника

В первом эксперименте все **100000 из 100000** сформированных точек оказались внутри треугольника. Теоретический центроид треугольника равен
$(1.666667, 2.000000, 0.666667)$,
а эмпирический центроид выборки составил
$(1.664533, 2.000911, 0.667275)$.

Средние значения барицентрических координат получились следующими:

- $\lambda_1 = 0.333639$,
- $\lambda_2 = 0.332724$,
- $\lambda_3 = 0.333637$.

Эти значения близки к теоретически ожидаемому значению $1/3$.

Распределение точек по 4 равновеликим частям треугольника оказалось следующим:

- область 1–25100 точек;
- область 2–24885 точек;
- область 3–25061 точек;
- область 4–24954 точки.

Ожидаемое число точек в каждой области составляло **25000**.

2. Равномерное распределение точек внутри круга

Во втором эксперименте все **100000 из 100000** сформированных точек оказались внутри круга. Среднее значение величины $(r/R)^2$составило **0.500481**, что практически совпадает с теоретическим значением **0.500000**.

Распределение точек по 8 равновеликим областям круга оказалось следующим:

- область 1–12421 точка;
- область 2–12428 точек;
- область 3–12637 точек;
- область 4–12391 точка;
- область 5–12571 точка;
- область 6–12558 точек;
- область 7–12455 точек;
- область 8–12539 точек.

Ожидаемое число точек в каждой области составляло **12500**.

3. Равномерное распределение направлений на единичной сфере

В третьем эксперименте было сформировано **100000 из 100000** направлений. Средняя длина вектора составила **1.000000**, что полностью совпадает с теоретическим значением. Среднее значение координаты zсоставило **0.001705**, что близко к ожидаемому значению **0**.

Распределение направлений по 8 равновеликим областям сферы оказалось следующим:

- область 1–12415 направлений;

- область 2–12641 направлений;
- область 3–12482 направления;
- область 4–12370 направлений;
- область 5–12510 направлений;
- область 6–12571 направление;
- область 7–12444 направления;
- область 8–12567 направлений.

Ожидаемое число направлений в каждой области составляло **12500**.

4. Косинусное распределение направлений относительно нормали

В четвертом эксперименте было сформировано **100000 из 100000** направлений, причем все **100000** направлений оказались в полусфере относительно нормали N. Средняя длина вектора составила **1.000000**. Среднее значение $\mu = \cos(\theta)$составило **0.667006**, что близко к теоретическому значению **0.666667**. Среднее значение μ^2составило **0.500098**, что практически совпадает с ожидаемым значением **0.500000**.

Распределение направлений по 8 равновероятным областям оказалось следующим:

- область 1–12515 направлений;
- область 2–12517 направлений;
- область 3–12453 направления;
- область 4–12496 направлений;
- область 5–12521 направление;
- область 6–12549 направлений;
- область 7–12511 направлений;
- область 8–12438 направлений.

Ожидаемое число направлений в каждой области составляло **12500**.

Анализ результатов вычислений

Полученные результаты показывают, что реализованные в программе методы формирования случайных величин работают корректно. Во всех четырех экспериментах было сформировано ровно **100000** точек или направлений, что соответствует условию задания. Кроме того, программа завершила работу без ошибок, а все рассчитанные статистические характеристики оказались близки к теоретически ожидаемым значениям.

В первом эксперименте все **100000** точек оказались внутри треугольника. Эмпирический центроид выборки $(1.664533, 2.000911, 0.667275)$оказался очень близок к теоретическому центроиду $(1.666667, 2.000000, 0.666667)$. Средние значения барицентрических координат $\lambda_1 = 0.333639$, $\lambda_2 = 0.332724$, $\lambda_3 = 0.333637$практически совпадают с ожидаемым значением $1/3$. Распределение точек по четырем равновеликим частям также оказалось близким к равномерному: отклонения от ожидаемого значения **25000** точек невелики. Это подтверждает, что точки действительно формируются равномерно внутри треугольника.

Во втором эксперименте все **100000** точек оказались внутри круга, что подтверждает корректность построения точек в заданной плоскости и в пределах радиуса. Среднее значение $(r/R)^2 = 0.500481$ практически совпадает с теоретическим значением **0.5**, которое должно наблюдаться при равномерном распределении точек по площади круга. Распределение по 8 равновеликим областям также оказалось близким к равномерному: числа попаданий отличаются от ожидаемых **12500** незначительно. Следовательно, можно сделать вывод, что распределение точек внутри круга является равномерным.

В третьем эксперименте было сформировано **100000** направлений на единичной сфере, то есть направление было получено для каждой выборки. Средняя длина вектора равна **1.000000**, что подтверждает принадлежность всех направлений единичной сфере. Среднее значение координаты $z = 0.001705$близко к нулю, как и должно быть при равномерном распределении по поверхности сферы. Распределение направлений по 8 равновеликим областям также оказалось практически равномерным. Это подтверждает корректность метода генерации равномерных направлений на сфере.

В четвертом эксперименте было сформировано **100000** направлений, и все они оказались в полусфере относительно нормали N. Средняя длина вектора равна **1.000000**, что подтверждает единичную длину всех направлений. Среднее значение $\mu = \cos(\theta)$составило **0.667006**, что очень близко к теоретическому значению **2/3**. Среднее значение $\mu^2 =$

0.500098практически совпадает с ожидаемым значением **0.5**. Распределение направлений по 8 равновероятным областям также близко к равномерному. Следовательно, реализованный алгоритм действительно формирует косинусное распределение направлений относительно нормали.

Таким образом, результаты вычислительного эксперимента подтверждают, что все реализованные методы работают правильно: точки в треугольнике и круге распределяются равномерно, направления на сфере формируются равномерно, а направления в четвертом эксперименте подчиняются косинусному закону распределения.

Вывод

В ходе выполнения лабораторной работы были изучены методы формирования случайных точек и направлений в заданных областях пространства. Были реализованы алгоритмы равномерного распределения точек внутри треугольника и круга, равномерного распределения направлений на единичной сфере, а также формирования направлений с косинусным распределением относительно заданной нормали.

По результатам вычислительного эксперимента установлено, что все **100000** выборок в каждом случае были сформированы корректно. Полученные статистические характеристики оказались близки к теоретически ожидаемым значениям, что подтверждает правильность реализованных методов и соответствие программы условиям задания.

Таким образом, цель лабораторной работы достигнута: методы формирования заданных распределений случайных величин были успешно реализованы и проверены на практике.